**Email:** John.dixon@hull.ac.uk
**Meetings:** https://tinyurl.com/jdixonhull
**Feedback:** https://tinyurl.com/jdhullfeedback
**Office:** RBB-315

# Software Design Patterns

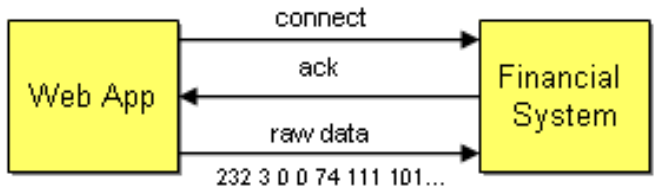
## Distributed Systems Programming

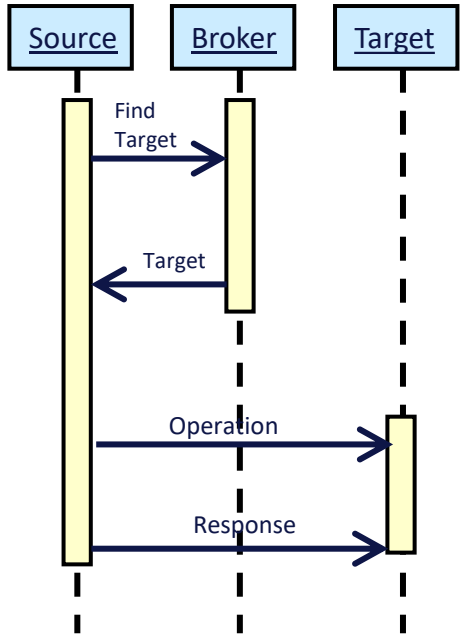
Dr John Dixon

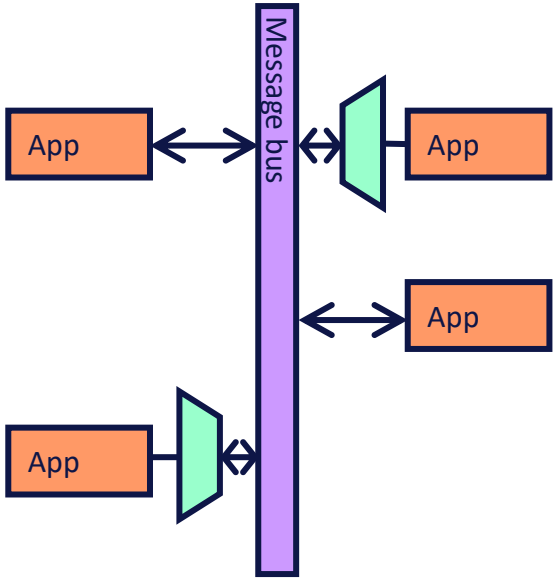
UNIVERSITY of HULL

# Recap: Message-based Design Patterns

- Point to point
  - simple
- Broker
  - Hides remote implementation details
- Message bus
  - Universal connector




UNIVERSITY of HULL

# Software Design Patterns

- Fit into three main groups:
  - Creational
    - *Abstract factory*, Builder, Factory method, Prototype, *Singleton*
  - Structural
    - Adapter, Bridge, Composite, Decorator, *Façade*, Flyweight, *Proxy*
  - Behavioural
    - Chain of responsibility, *Command*, Interpreter, Iterator, Mediator, Memento, *Observer*, State, Strategy, Template method, Visitor

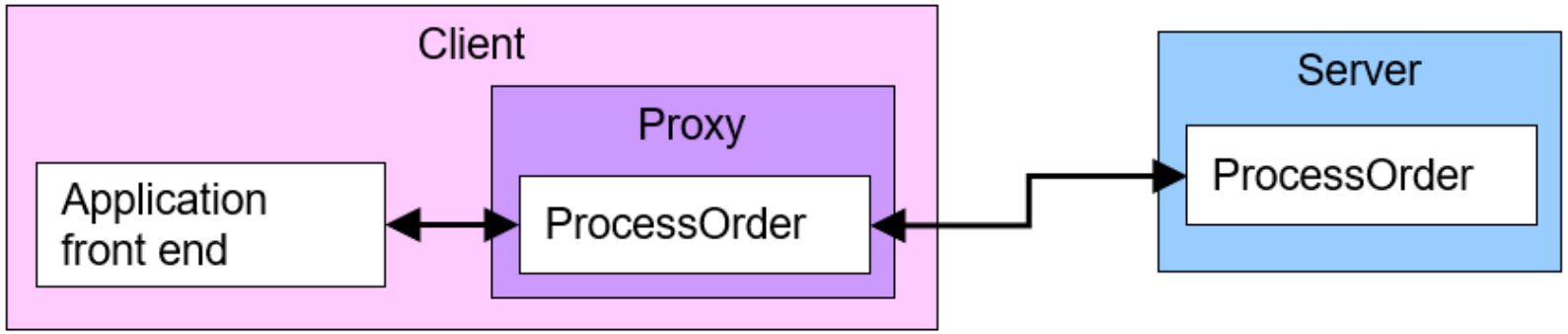
We will be considering some of the most common

- But again – read around these areas as they are useful
  - And practical implementation helps make patterns more clear

UNIVERSITY of HULL

# Proxy pattern

- Provides a surrogate or placeholder for another object to control access to it
  - Instantiate real object
  - Remember identity of real object
  - Forward requests to real object
  - Forward replies from real object
- Client  believes it is communicating directly with endpoint




- In which common situations can you use the proxy pattern?

UNIVERSITY of HULL

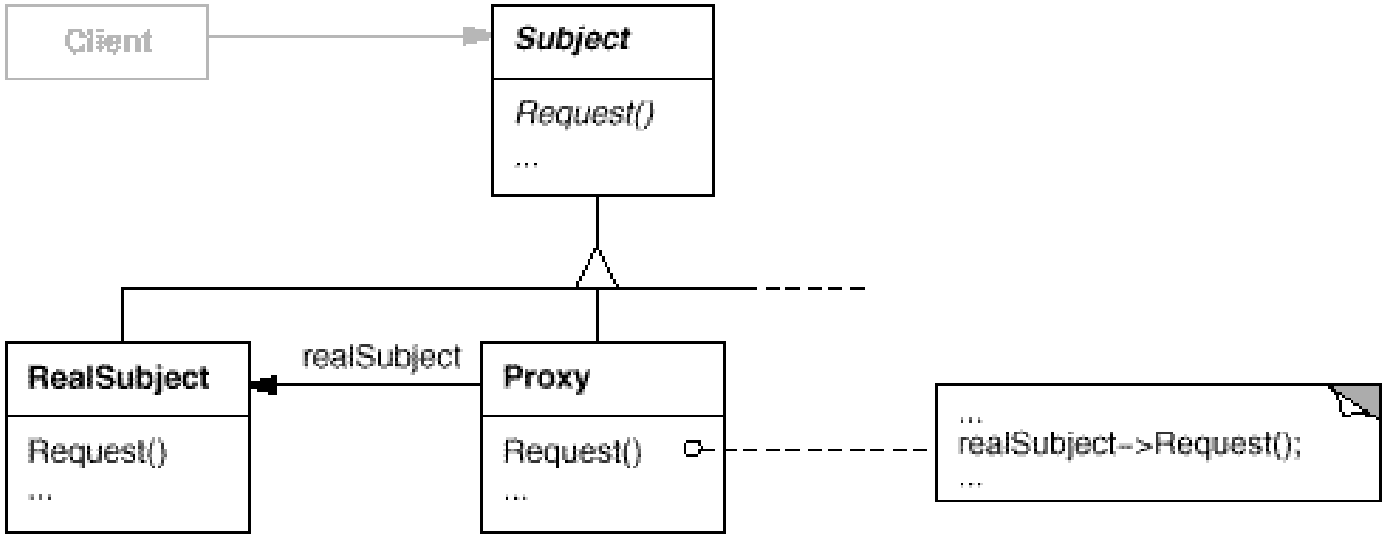
# Proxy pattern: Common uses

- A **virtual proxy** is a placeholder for "expensive to create" objects
  - Real object is *only* created when a client first requests/accesses the object.
- A **remote proxy** provides a local representative for an object that resides in a different address space
  - "stub" code in RPC and CORBA, proxy in web services – means client can make calls as if remote object is local
- A **protective proxy** controls access to a sensitive master object
  - Surrogate object checks that the caller has the access permissions required prior to forwarding the request.
- A **cache proxy** provides temporary storage of the results of expensive target operations
  - multiple clients can share the results.
- **Smart proxy** interposes additional actions when an object is accessed.
  - Counting the number of references to the real object so that it can be freed automatically when there are no more references (aka smart pointer)
  - Loading a persistent object into memory when it's first referenced
  - Checking that the real object is locked before it is accessed to ensure that no other object can change it.

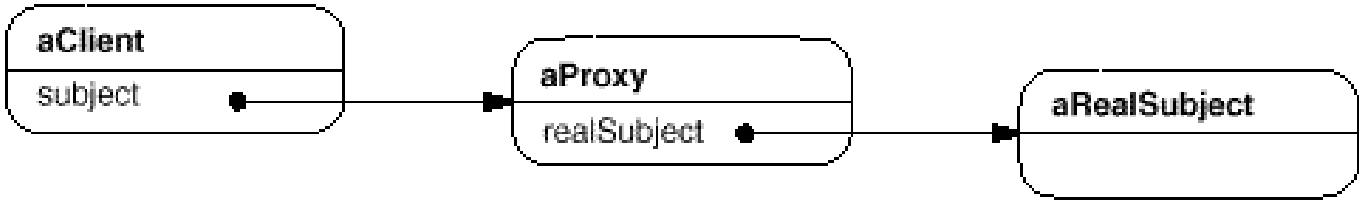
UNIVERSITY of HULL

# Proxy pattern: Class diagram

- Classes:




- Objects:



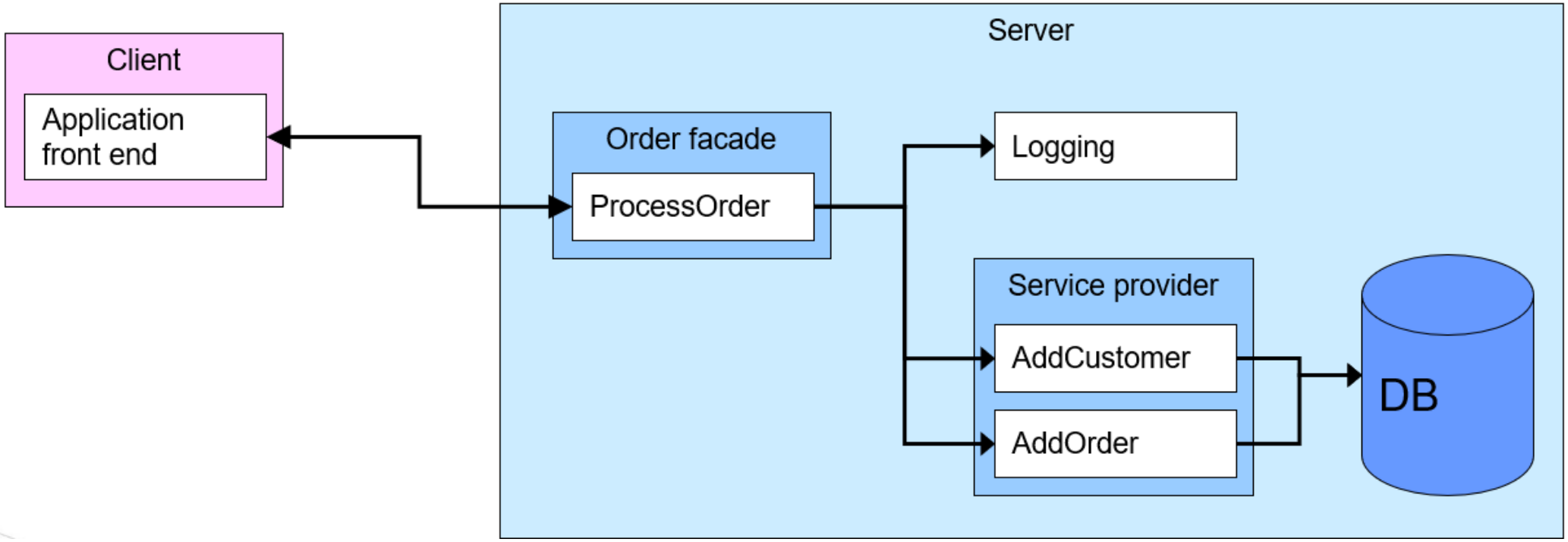

UNIVERSITY of HULL

# The Façade Pattern

- Façade
  - Layer between service provider and consumer
  - Encapsulates a complex task which uses several service provider services.
    - Providing a single source of access
- Useful for
  - Encapsulating functionality/data of legacy systems
  - Moving business logic away from client
  - Implementing interception services (logging, error-checking)
  - Adding transactions to high-level tasks involving multiple database operations
  - Service-oriented design: aggregate fine-grained methods in single service call

# Façade example

- Using a façade
  - Encapsulates higher level process
  - Simplifies client code
  - Moves business logic to façade running on server
  - Can bring together multiple server functions
  - May encapsulate functions from different server systems



# Order Processing Example

- Consider the following code, using a service provider:

```
// Client code
void SubmitOrder() {
  CustomerDetails cust = new CustomerDetails();
  OrderDetails order = new OrderDetails();
  CustomerDatabase custDB = new CustomerDB();
  OrderDatabase ordDB = new OrderDB();

  if (!custDB.Exists(cust))
  {
     custDB.AddCustomer(cust);
  }

  ordDB.AddOrder(order);
}
```

**Client needs to do lots of work and is left in charge of database tidiness!**

UNIVERSITY of HULL

# Order Processing Example

```
// Facade code
public class Facade {
  public void ProcessOrder(Customer cust, Order ord)
  {
    CustomerDatabase custDB = new CustomerDB();
    OrderDatabase ordDB = new OrderDB();
    if (!custDB.Exists(cust))
    {
      custDB.AddCustomer(cust);
    }
    ordDB.AddOrder(order);
  }
}
```

Façade does all the work

Client code is more simple

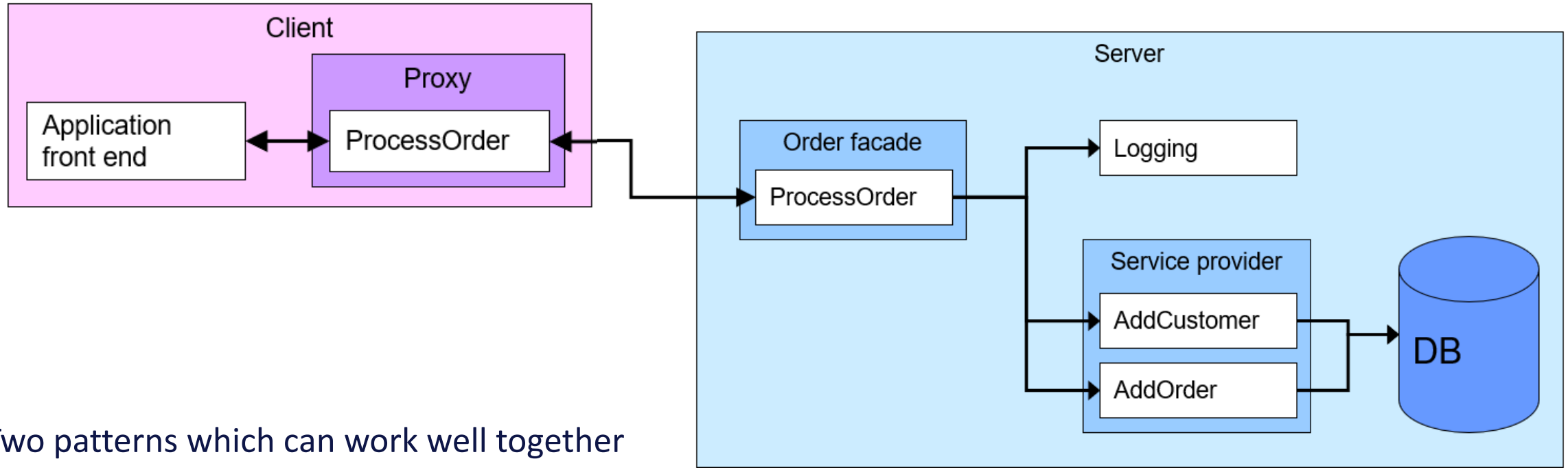
Client doesn't have to worry about the implementation details

```
// Client code
void SubmitOrder() {
  CustomerDetails cust = new CustomerDetails();
  OrderDetails order = new OrderDetails();
  Façade fac = new Façade();

  fac.ProcessOrder(order, cust)
}
```

UNIVERSITY of HULL

# Façade + Proxy



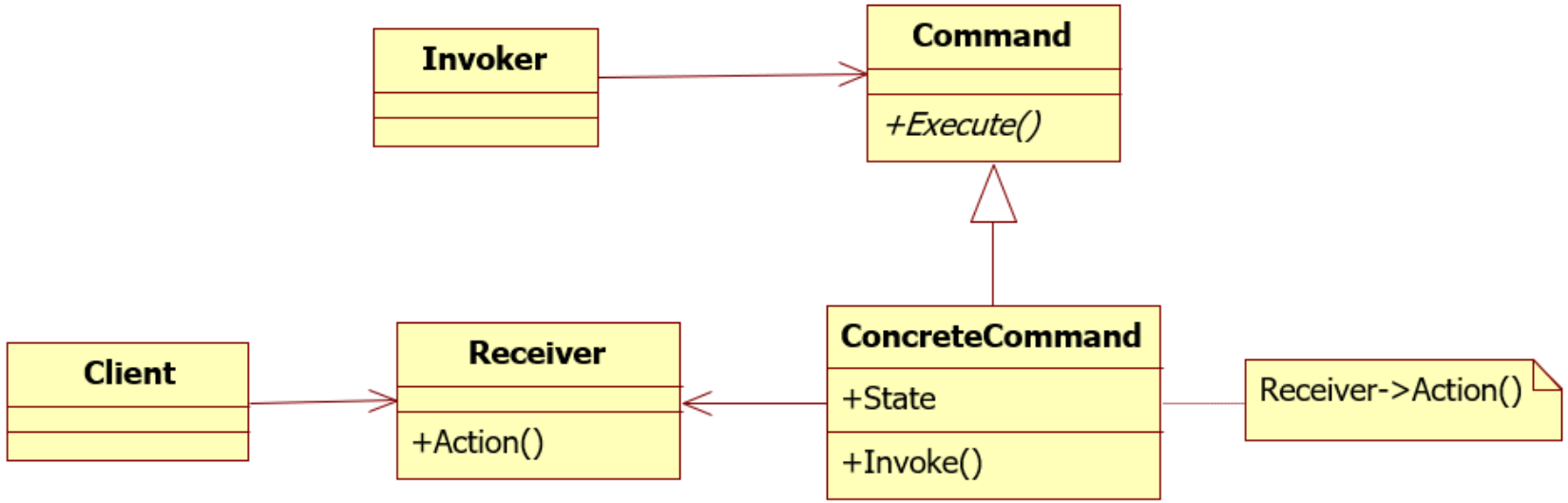

Two patterns which can work well together

Underlines that patterns help with problems but multiple problems may exist so multiple patterns can be used where they don't clash.

UNIVERSITY of HULL

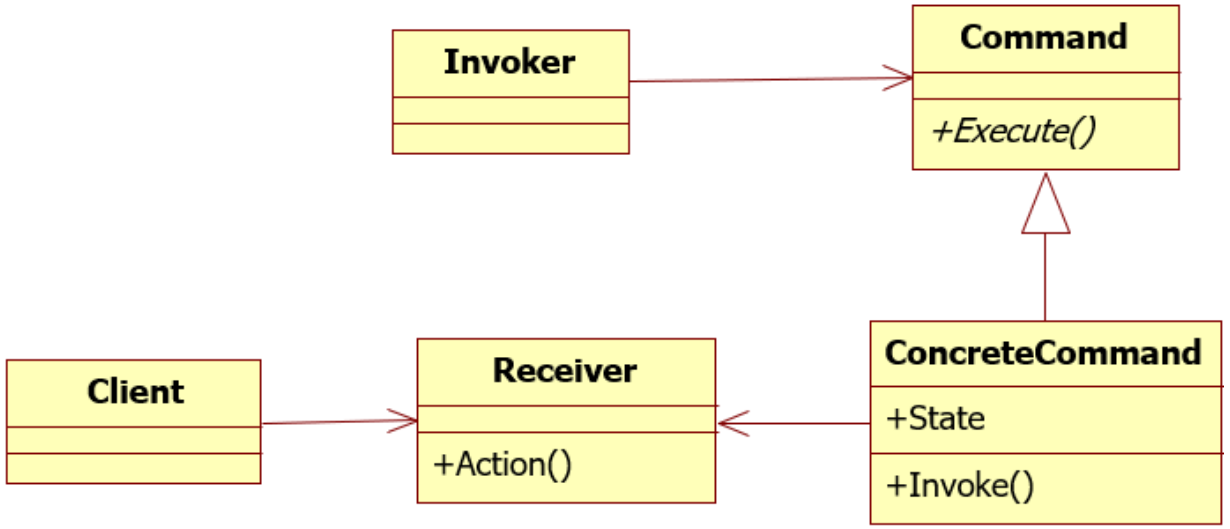
# Command pattern

- Encapsulates action requests as objects rather than calls
  - Allows client to issue commands without knowing recipient
  - Client creates command object based on an interface
  - Command object has knowledge of method to be invoked




UNIVERSITY of HULL

# Command pattern: Example




- Client is **Customer** in a restaurant
- Invoker is **Waiter**
- Receiver is **Cook** (actually does the work)
- Commands are **Orders**
- Concrete commands are the **Tickets** sent to the kitchen

- Customer wants to pass Order directly to Cook
- But Cook has lots of work to do and Customer doesn't know exactly how to format the Ticket
- So Customer passes its order through the Waiter, who converts it to a Ticket and puts it at the back of the queue
- Cook now receives Ticket when items earlier in the queue are dealt with

# Distributed Command pattern

- Sending commands over the network
  - Client does not have to ‘execute’ the functionality locally
  - Instead – the invoker can send the command over the network to a receiver on another machine
  - The receiver does the actual work
  - The result is passed back to the client

- Or…
  - Client sends a message over network
  - Server receives message and puts it on queue
  - Invoker runs as service and polls queue for new messages
  - Invoker calls Execute() on each command object
  - receiver passes the result back to client over network

- This pattern works well with:
  - Message queuing (commands can be queued)
  - A proxy
    (e.g. invoker knows WHAT command to invoke but not WHERE the receiver is – proxy manages the WHERE)

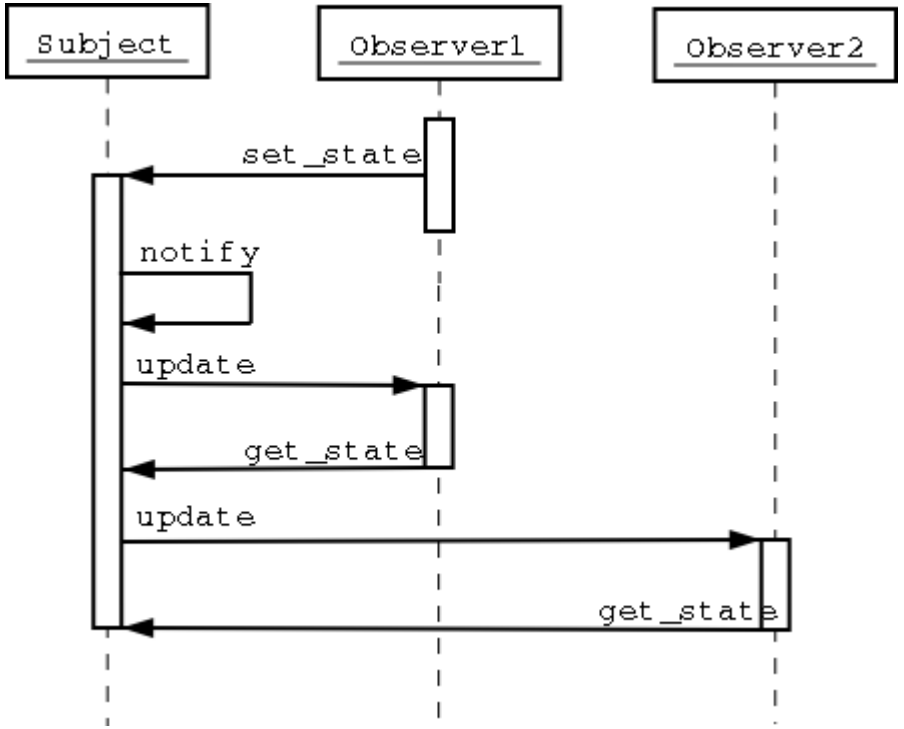
UNIVERSITY of HULL

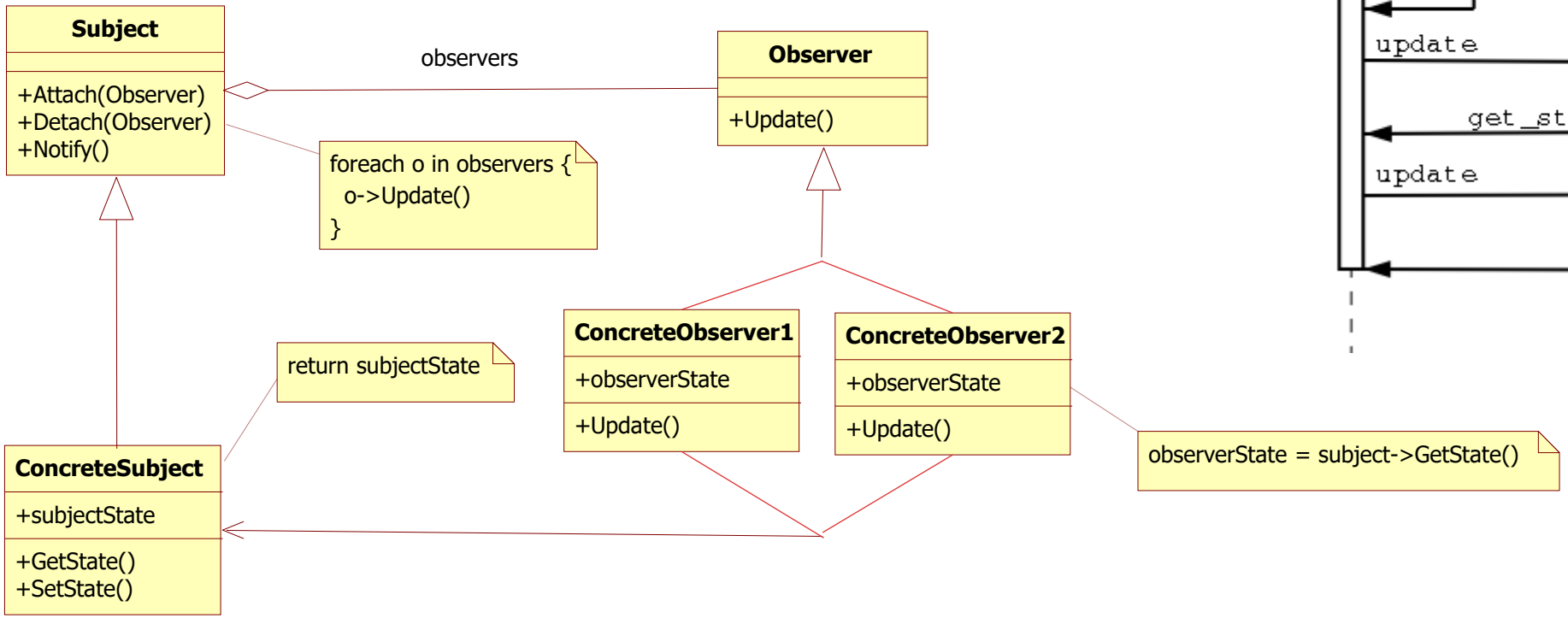
# Observer pattern (publish/subscribe)

- Use the Observer pattern to maintain a list of interested dependents (observers) in a separate object (the subject).
- Have all individual observers implement a common Observer interface to eliminate direct dependencies between the subject and the dependent objects
- Consider an auction. Each person is an observer, raised paddles indicate an update. The auctioneer broadcasts the update each time a paddle is raised. Each person receives the update and uses it to make the next decision.

# Observer: Class diagram

- Can operate as a PUSH (broadcast) or PULL (request)
- Observer and Subject have clear interfaces
- Useful for
  - Event handling (including distributed – e.g. feeds)



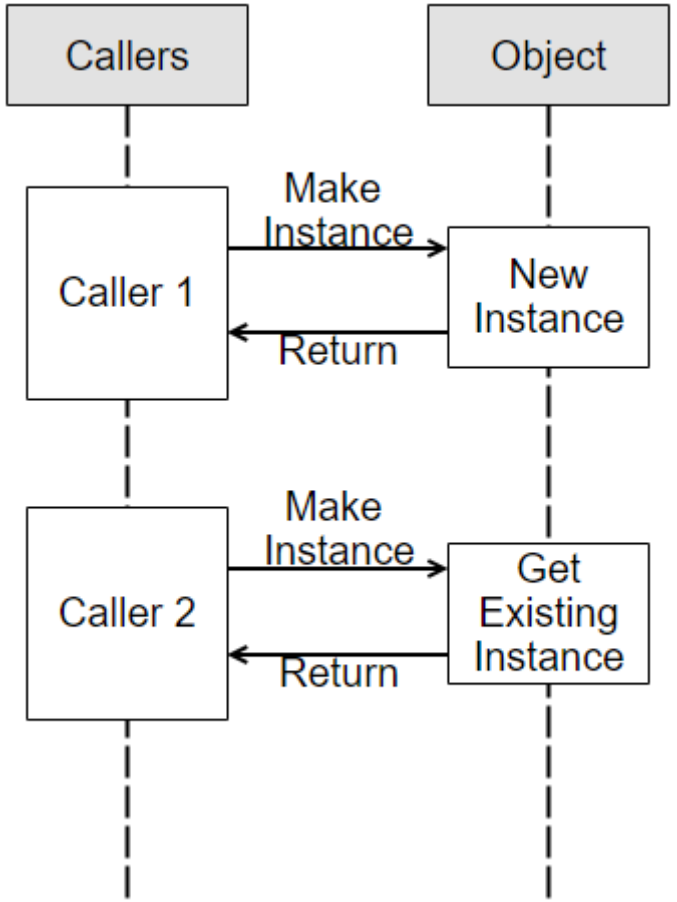

UNIVERSITY of HULL

# Singleton

Sometimes you only want one version of an Object to exist at runtime but you can't always pass that single Object around as (e.g.) parameters.

*Singleton* provides a global, single instance by:

- Making the class create a single instance of itself.
- Allowing other objects to access this instance through a class method that returns a reference to the instance. A class method is globally accessible.
- Declaring the class constructor as private so that no other object can create a new instance.




UNIVERSITY of HULL

# Singleton pattern

- Implements a class which can have only one instance
  - Or sometimes a fixed number of instances (not a pure singleton)
- Implemented by creating a class with a method that creates a new instance of the object if one does not exist
  - Lazy allocation
- If an instance already exists, it returns a reference to that object
- Uses
  - Resource reduction for server objects
  - Central resource management (audio managers; gateway that manages connections to legacy system)
  - Database access (Entity Framework)
    - You wouldn't want lots of instances of the same database data as they can easily get out of sync
- Care needed in multi-threading
  - Mutex (mutual exclusion – allowing only one thread to access the resource) required on instantiation class
  - (we'll consider this in a later lecture)

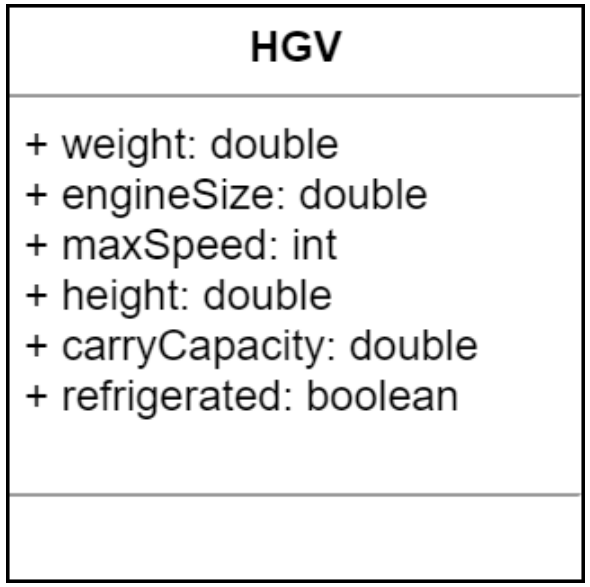
| Singleton |
| --- |
| -singletonInstance<br>... |
| «constructor»<br>-Singleton( )<br>«misc»<br>+getInstance( )<br>... |

UNIVERSITY of HULL

# Creational : Factory Design Pattern

Let’s assume we want to write some software for a vehicle insurance company.

They start out offering insurance for HGVs so a serverside class like this might make sense:




UNIVERSITY of HULL

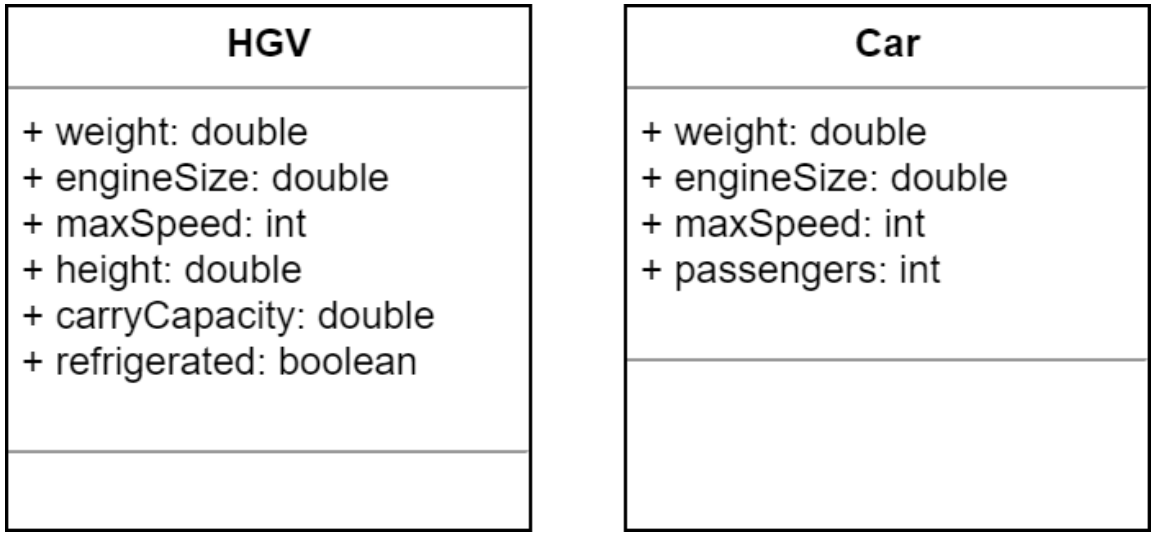
# Creational : Factory Design Pattern

Now our insurance company wants to also insure cars.

We can define another class to define cars.

But when a person tries to arrange insurance our software doesn't know which type of vehicle object it will need to create **at runtime** because they could want to insure a car or an HGV.

We could make large modifications to our server software each time we want to add a new type of vehicle we want to insure - but that's not time or money efficient.

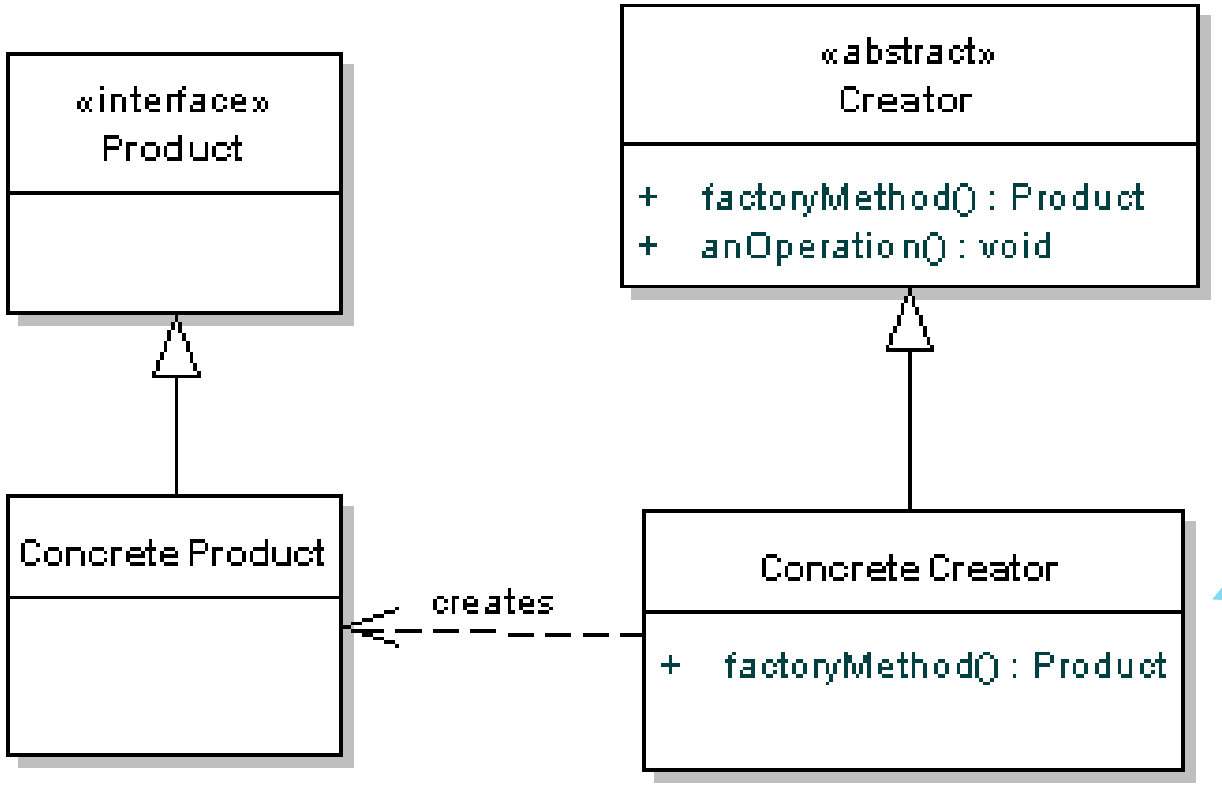



UNIVERSITY of HULL

# Creational : Factory Design Pattern

The factory pattern allows us to generate the object that is right for the particular situation at runtime without needing to know ahead of time.

- It also allows us to easily make more types of object available later if, for example, our insurer wanted to start insuring bikes too.




We use the concrete creator to make concrete products.

The rest of our software only cares about the Creator and Product **interfaces**

So when we want to make new vehicles we just add a new concrete product. The concrete creator can now make one.

UNIVERSITY of HULL

# Creational : Factory Design Pattern

```
public interface IVehicle
{
    double weight;
    double engineSize;
    int maxSpeed;
}
public class HGV : IVehicle
{
    double weight;
    double engineSize;
    int maxSpeed;
    double height;
    double carryCapacity;
    bool refrigerated;
}
public class Car : IVehicle
{
    double weight;
    double engineSize;
    int maxSpeed;
    int passengers;
}
```

```
public interface IVehicleFactory
{
    public IVehicle GetVehicle(string Vehicle);
}

public class ConcreteVehicleFactory : IVehicleFactory
{
    public IVehicle GetVehicle(string Vehicle)
    {
        switch (Vehicle)
        {
            case "HGV":
                return new HGV();
            case "Car":
                return new Car();
            default:
                throw new ApplicationException(
                 string.Format(
                 "Vehicle '{0}' cannot be created", Vehicle));
        }
    }
```

UNIVERSITY of HULL

# Creational : Factory Design Pattern

Using our factory the client doesn't need to know what we are going to create or how

- It knows that the Interface rules for factory and vehicle will be upheld

In this example we base our object creation on a ReadLine() – (Although realistically this needs verification)

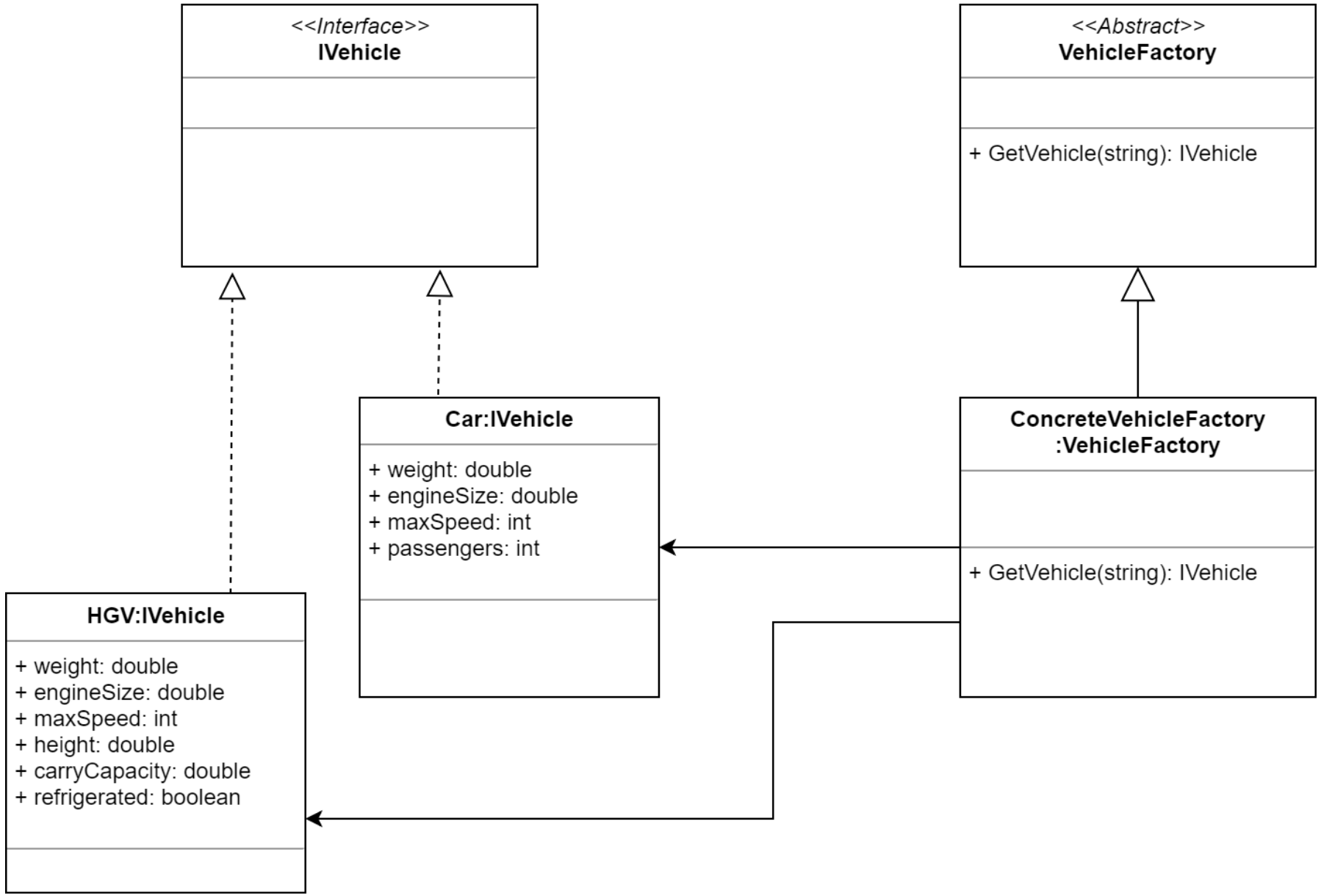
```
IVehicleFactory factory = new ConcreteVehicleFactory();

IVehicle myVehicle = factory.GetVehicle(Console.ReadLine());
```

The concrete factory can be swapped out at any time with minimal effort.
New vehicles can be added at any time with minimal effort.

Concrete factory can be made even more powerful by creating objects at runtime through mechanisms such as reflection
…no need to update the concrete factory…

# Creational : Factory Design Pattern




UNIVERSITY of HULL

# The Factory Pattern

- Factory
  - Class which abstracts object creation process and creates typed object instances
  - Can be swapped out with a different implementation easily
- Useful with Remote Procedure Calls
  - Client uses factory to create remote object
  - Based on configuration file settings, Factory decides whether to
    - Create local object instance
    - Return proxy to remote object

# Summary

- Design patterns
  - Encapsulate best practice
  - Help prevent reinventing the wheel
  - Encourage modular, reusable, maintainable code
- Several useful design patterns for distributed systems
  - Broker
  - Message Bus
  - Proxy
  - Façade
  - Observer
  - Command
  - Singleton
  - Factory
- Use the books/online resources!!

UNIVERSITY of HULL

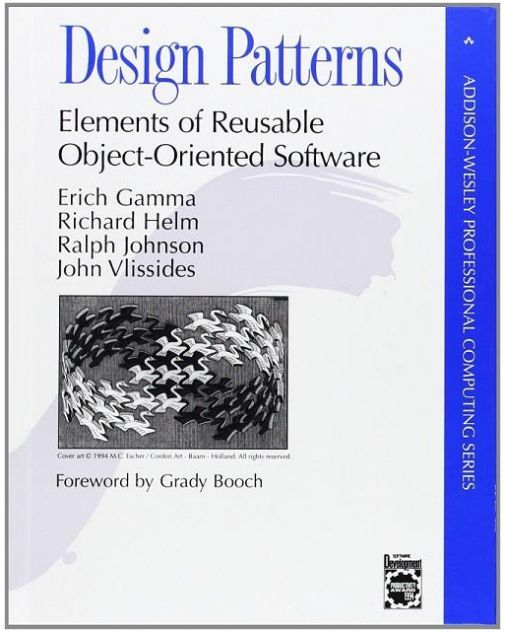
# Books

**Design Patterns:**

**Elements of Reusable**

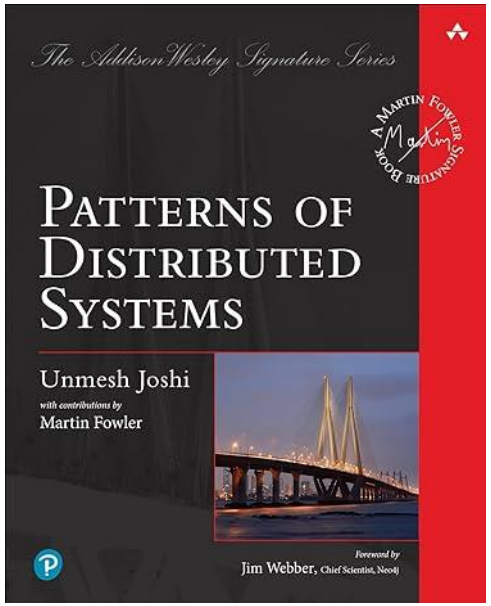
**Object-Oriented Software**

Erich Gamma, Richard Helm,
Ralph Johnson, John Vlissides



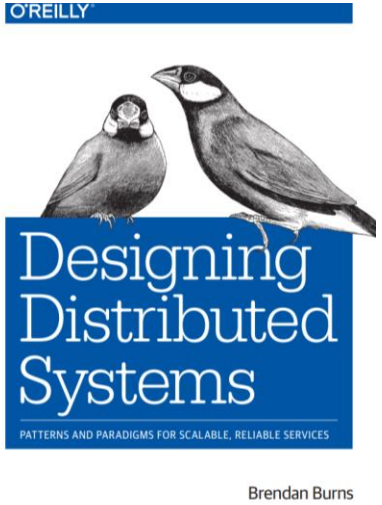

**Patterns of Distributed Systems**

Unmesh Joshi




**Designing Distributed Systems**

Brendan Burns




Free:

https://info.microsoft.com/rs/157-GQE-382/images/EN-CNTNT-eBook-DesigningDistributedSystems.pdf

UNIVERSITY of HULL

# Reflection

We've looked at:

- Classic Software Design Patterns for distributed systems

In the next session we will continue to look at Services and Service Oriented Architectures

UNIVERSITY of HULL